DATA MANAGEMENT PROJECT — 2026 SEASON

Formula 1 2026

A Multi-Source Data Management Pipeline

Acquisition · Storage · Cleaning · Integration · Quality Assessment · Analysis

Filippo Avon — Matricola 886229

Università degli Studi di Milano-Bicocca | Data Management | September 2026

Presentation

A short abstract of the domain and of the project, before we go into the technical detail.

THE DOMAIN

Formula 1 in Brief

- The top tier of open-wheel motorsport: 10 teams, roughly 20 drivers, about 24 Grands Prix per season worldwide
- Every session (practice, qualifying, race) generates dense timing, telemetry and strategy data — lap times, pit stops, tyre choices, car sensor readings sampled several times per second
- A rich, real-world domain for data management: high volume, multiple independent official and community data sources, and meaningful relationships between drivers, teams, races and circuits

THE PROJECT

This Project in One Paragraph

- A complete data management pipeline built around the 2026 F1 season, combining a relational dataset (historical results) and a semi-structured dataset (session telemetry) from two independent public sources
- Covers the full lifecycle: acquisition, storage (MySQL + MongoDB), cleaning, integration & enrichment, data quality assessment, and analysis through four research questions
- Designed to demonstrate not just “getting an answer”, but doing so with measured, documented data quality at every step

Project at a Glance

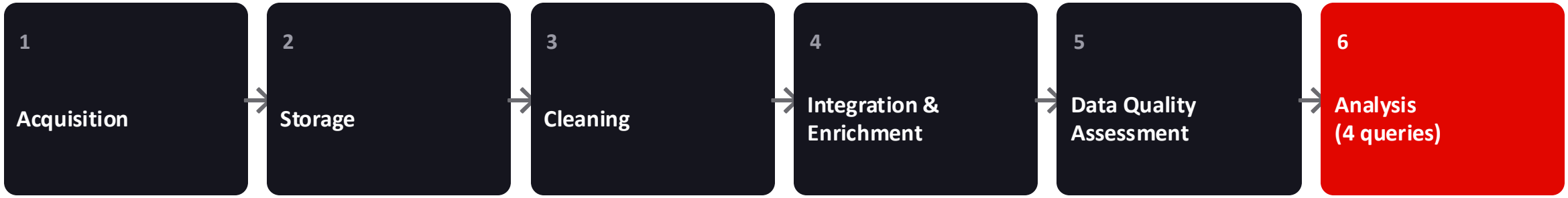
A complete, reproducible pipeline combining a relational and a semi-structured data source for the 2026 F1 season.

2	47,771	2	100%	4
data sources	raw files acquired	DBMS (MySQL + MongoDB)	driver & race match rate	research questions answered

Research Questions

1. RQ1 — Fastest pit stop of the 2026 season?
2. RQ2 — Highest peak speed recorded in 2026?
3. RQ3 — Best race pace on MEDIUM tyres at Barcelona?
4. RQ4 — Best qualifying-to-race “comeback”?

The Pipeline: Six Stages



Every stage is implemented as an independent, idempotent Python script, orchestrated from a Jupyter notebook, with both databases running as reproducible Docker containers.

Design principle: two DBMS chosen to match the shape of each source — a relational store for Kaggle's tabular, foreign-key-linked data, and a document store for TracingInsights' heterogeneous, session-dependent JSON.

Two Independent Data Sources

RELATIONAL

Kaggle

jtrotman/formula-1-race-data

- Ergast-style CSV files: races, results, qualifying, lap_times, pit_stops, standings, drivers, constructors, circuits
- Full history: 1950 – 2026
- Downloaded via the official kagglehub Python library
- Filtered to the 2026 season during Data Cleaning

→ stored in **MySQL 8.0**

SEMI-STRUCTURED

TracingInsights

github.com/TracingInsights/2026

- Per-session JSON derived from FastF1: telemetry (~3.7 Hz), lap times, tyre compound, weather, race control, circuit corners
- 47,771 files across the 2026 season
- Retrieved via the GitHub REST API (Trees API + Contents API fallback)
- Parallel download, then consolidated during Data Cleaning

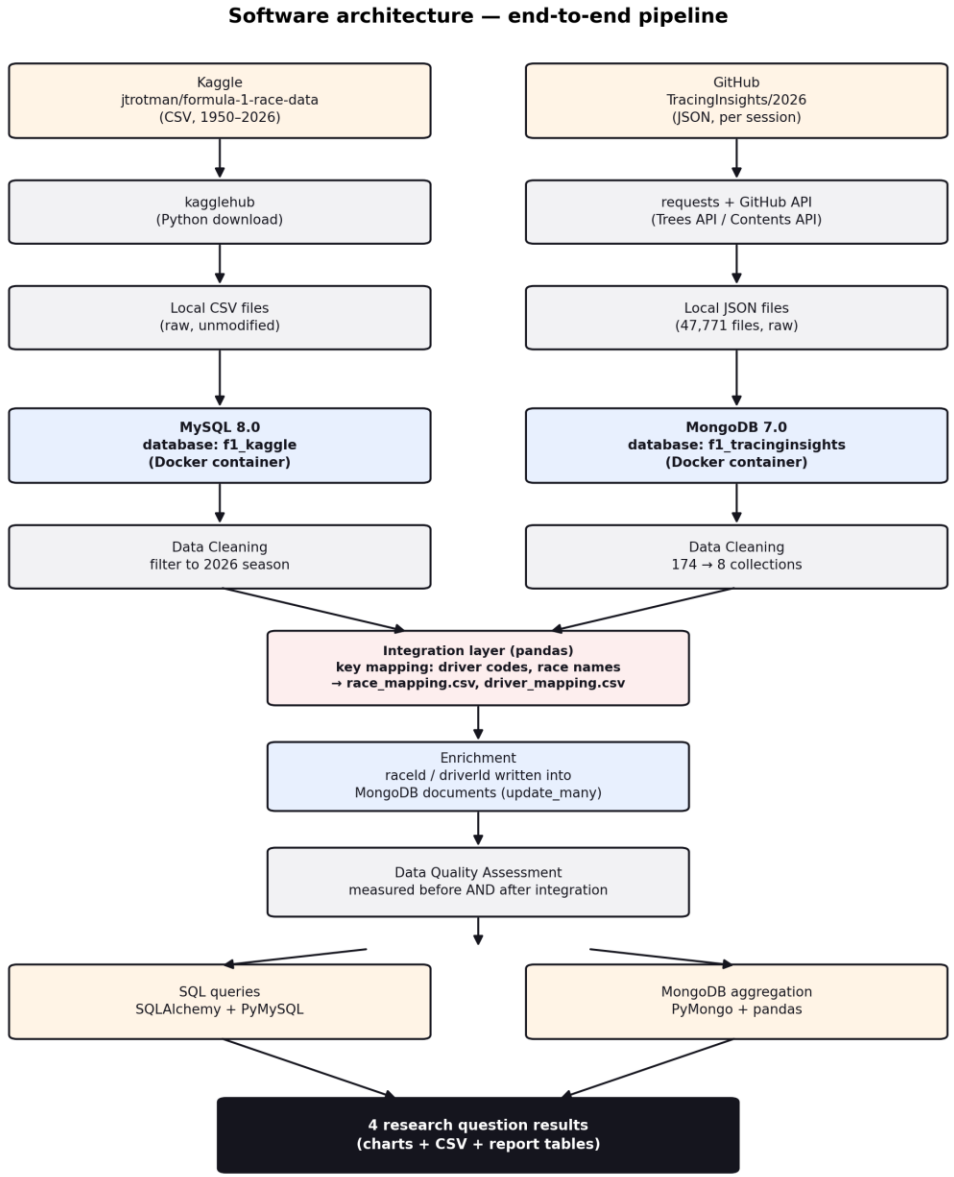
→ stored in **MongoDB 7.0**

Software Architecture

End-to-end pipeline, from source APIs to the four research-question results.

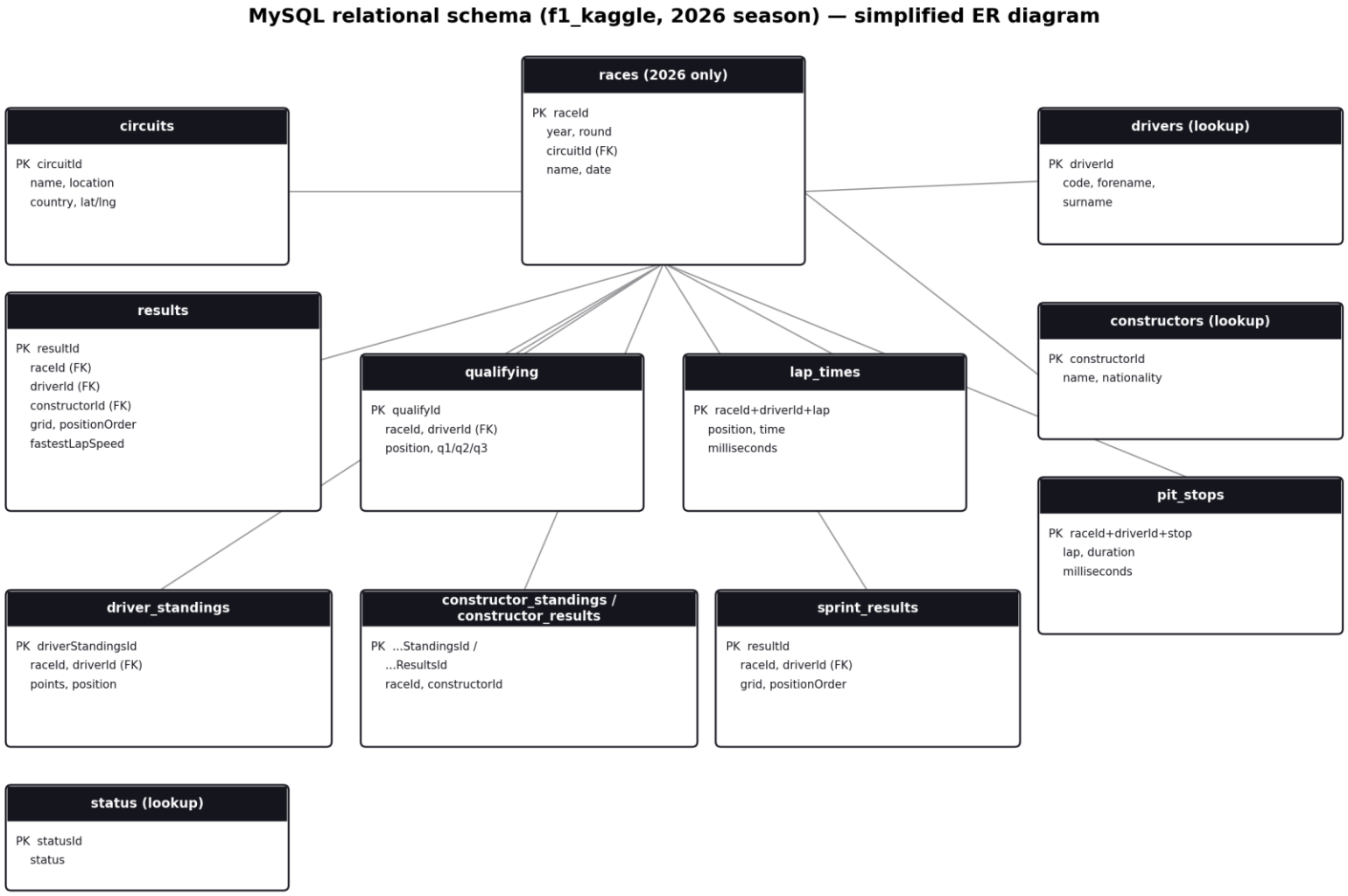
- Two source-specific acquisition scripts (kagglehub / GitHub API)
- Two Docker containers: MySQL 8.0 + MongoDB 7.0
- pandas-based integration layer building a key-mapping table
- Batched update_many() enrichment (no per-document writes)
- SQL (SQLAlchemy) and MongoDB aggregation for the 4 queries

Reproducibility: the entire storage layer is defined in a single docker-compose.yml — any evaluator can recreate an identical environment with one command.



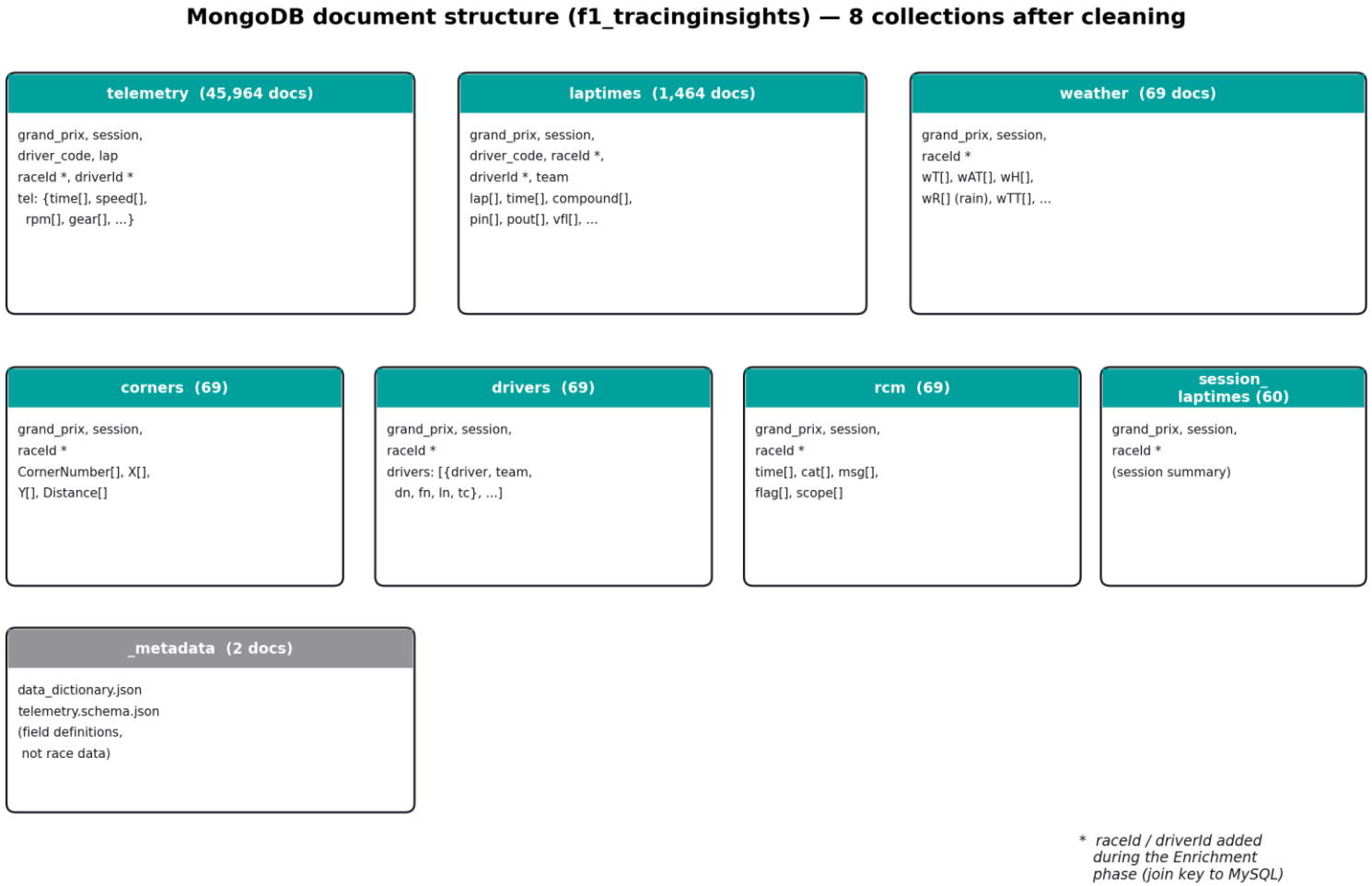
MySQL Relational Schema

f1_kaggle — 13 tables, filtered to the 2026 season (lookup tables kept in full)



MongoDB Document Structure

f1_tracinginsights — 8 collections after cleaning (down from 174 originally)



Data Cleaning: Two Issues Found & Fixed

MySQL — season scope

1950 – 2026

full history

→

2026 only

23 races

races.year identifies the relevant raceId values; the filter is propagated to every race-level table. Reference tables (drivers, constructors, circuits, status) are kept in full as join targets.

MongoDB — collection fragmentation

174

collections (1 per lap file)

→

8

clean collections

The original logic used each filename as the collection name, fragmenting telemetry into ~150 collections. Rewritten to route by path depth, consolidating all telemetry into one collection.

Integration & Enrichment

MySQL uses numeric surrogate keys; MongoDB uses human-readable identifiers. An automated mapping procedure reconciles the two — and its accuracy is explicitly measured.

12/12

races matched

11 direct + 1 manual mapping
("Barcelona" name variant)
3 testing sessions correctly excluded

23/23

championship drivers matched

100% direct match
9 reserve/test-only codes
correctly left unmatched

0

duplicate records found

MySQL results: 0 duplicates
MongoDB telemetry: 0 duplicates
(uniqueness check)

Key finding: every enrichment gap was traced to a documented, structural cause (pre-season testing sessions, reserve drivers) — not to an unexplained integration error. This distinction is the core deliverable of the quality assessment.

Data Quality Assessment

Measured explicitly both before and after integration — not just as a single after-the-fact percentage.

BEFORE INTEGRATION

Each source, independently

MySQL completeness

0% missing cells — all 9 tables

MySQL referential integrity

15/15 foreign keys 100% valid, 0 orphaned rows

MongoDB key completeness

17/17 checks 100% (grand_prix, session, driver_code, lap)

Uniqueness (both sources)

0 duplicates — MySQL results, MongoDB telemetry

AFTER INTEGRATION

Enrichment outcome & cross-source consistency

raceId coverage (MongoDB)

77.1% – 100%, depending on collection

driverId coverage (MongoDB)

98.7% – 99.0%

Race match rate

12/12 (100%) — 3 testing sessions excluded

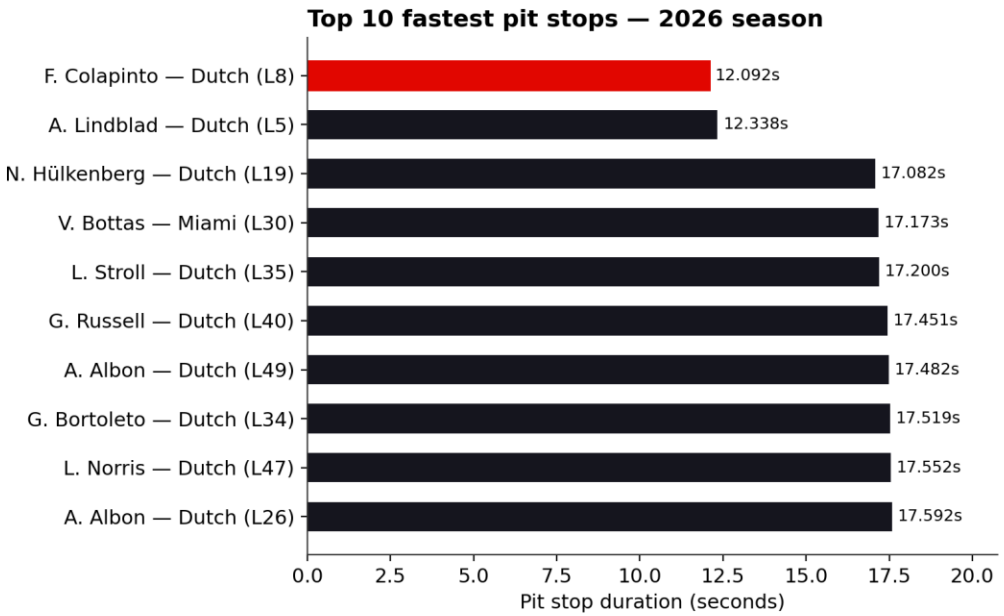
Driver match rate

23/23 championship drivers (100%)

Key insight: *both sources are independently complete and consistent, so every post-integration gap reflects differing source coverage — not a pre-existing quality defect.*

Results (1/2)

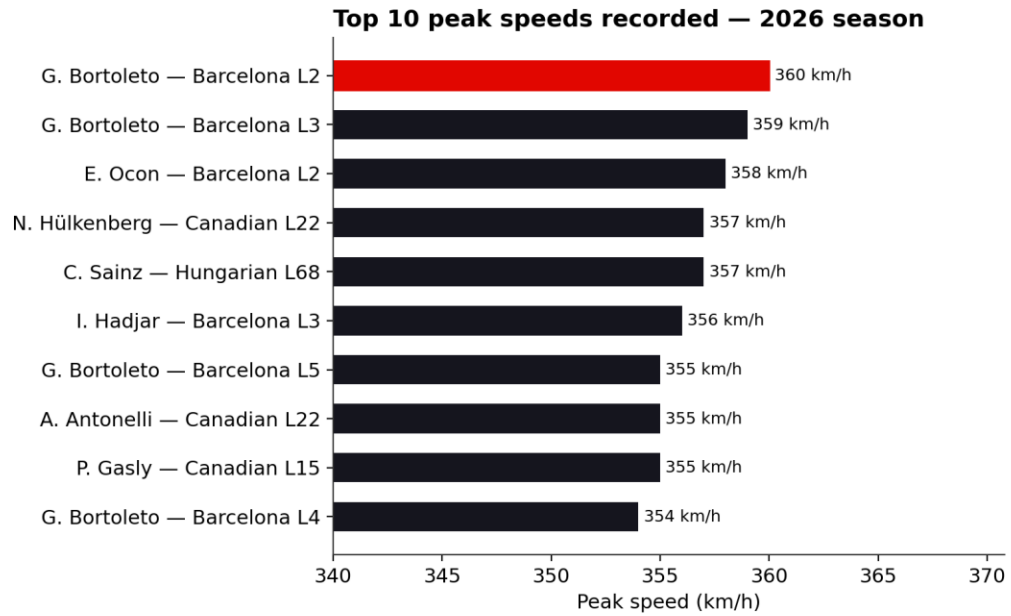
1 Fastest Pit Stop



Franco Colapinto (Alpine F1 Team)

12.092s at lap 8, Dutch Grand Prix

2 Highest Peak Speed

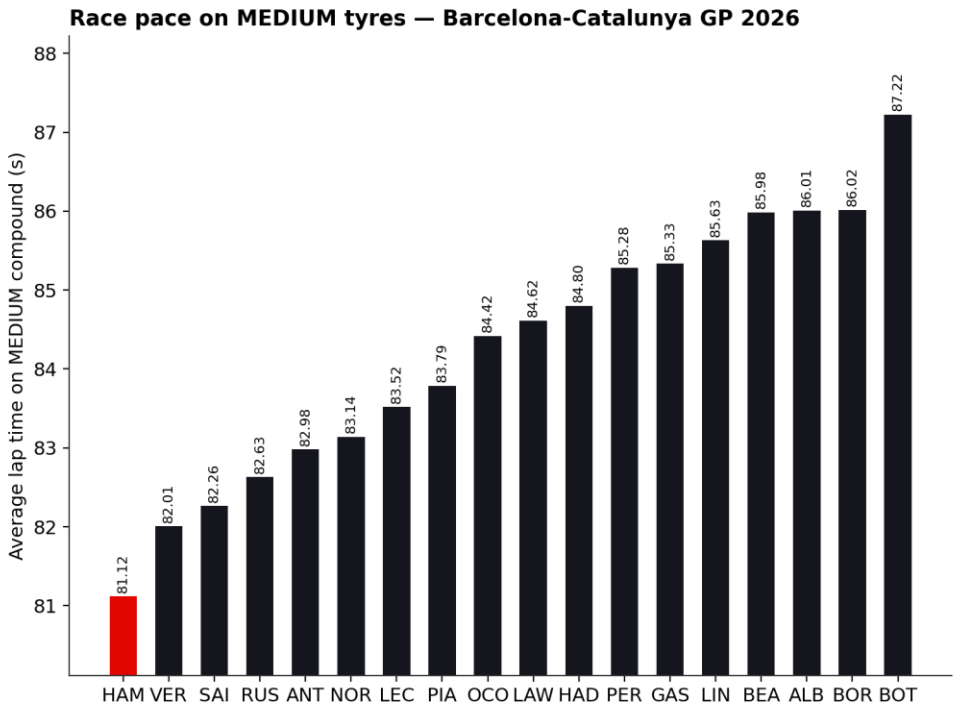


Gabriel Bortoleto (Audi)

360.0 km/h at lap 2, Barcelona-Catalunya GP

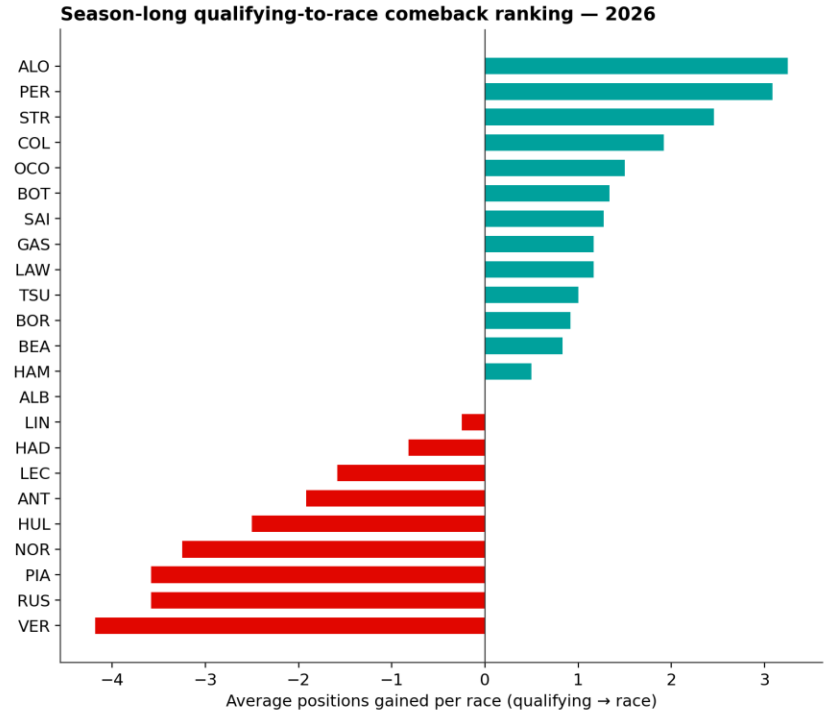
Results (2/2)

3 Best MEDIUM-Tyre Pace — Barcelona



Lewis Hamilton (Ferrari) — **81.117s** avg (12 clean laps)

4 Best Qualifying-to-Race Comeback



Fernando Alonso

+11 pos. (Monaco) **+3.25**/race avg

Limitations & Future Work

Limitations

- RQ1 pit stop duration reflects total pit-lane time (entry to exit), not the shorter stationary/tyre-change time alone (~2–3s in real F1) — not directly comparable to widely cited “fastest stop” records
- RQ3 pace comparison mixes drivers with very different clean-lap counts (4 to 37)
- Season still in progress at acquisition time — several races present only in MySQL
- One race-name discrepancy required a manually curated mapping rule

Future Work

- Re-run once the 2026 season is complete, for a full-season comeback ranking and peak-speed comparison
- Extend the spatial dimension: combine TracingInsights corner coordinates with MySQL circuits.lat/lng (e.g. MySQL spatial types or PostGIS)
- Automated outlier detection on raw telemetry before analysis
- Reusable, cross-season driver/race key-mapping lookup table

Conclusions

- ✓ A complete, reproducible pipeline across two DBMS chosen to fit each source's structure
- ✓ Integration accuracy explicitly measured: 100% of championship drivers and completed races matched
- ✓ Every quality gap traced to a documented cause, not left as an unexplained defect
- ✓ 4 research questions answered — 2 in pure SQL, 2 requiring MongoDB's telemetry-level detail — demonstrating the value of combining both sources

Thank you

Filippo Avon — 886229 — Data Management, Università degli Studi di Milano-Bicocca